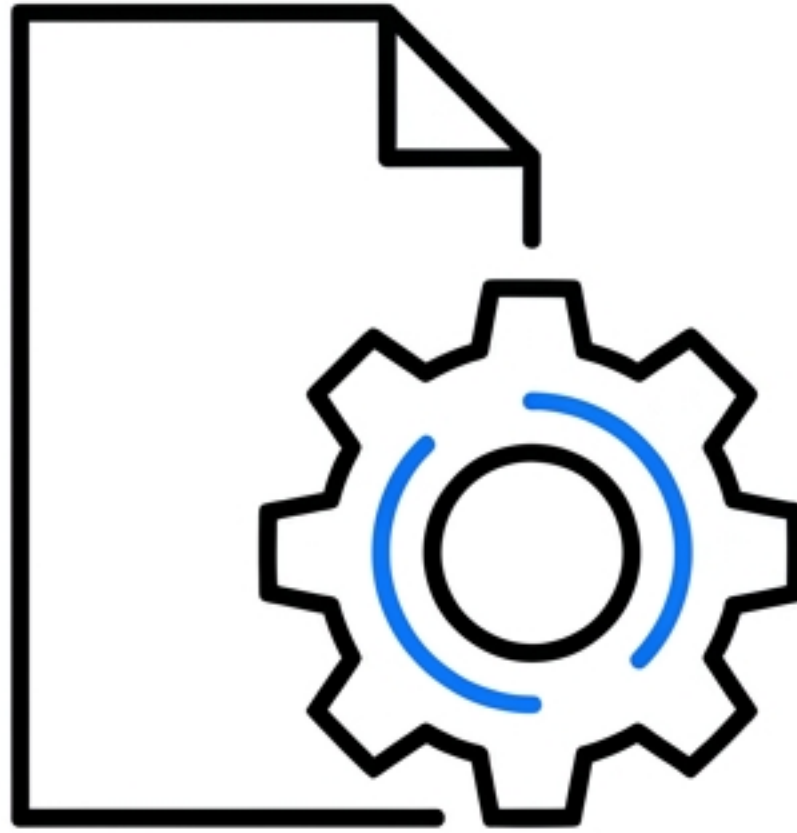




FICHEROS

GESTIÓN Y MANIPULACIÓN

CÓMO SON LOS FICHEROS



Un fichero es una secuencia de bytes almacenada en un medio no volátil.
Permite que los datos sobrevivan al cierre del programa.

ANATOMÍA DE UN FICHERO DE TEXTO



Archivos secuenciales: Los datos se guardan línea por línea. No se puede insertar texto en el medio sin reescribir.

TIPOS DE APERTURA: LOS MODOS

Antes de acceder, debes declarar tu intención al Sistema Operativo.

r



Lectura. Falla si no existe.

w



Sobreescritura. Borra todo el contenido previo.

a



Añadir. Agrega al final sin borrar.

r+



Lectura y Escritura simultánea.

APERTURA: MANUAL VS AUTOMÁTICO

MANUAL



Requiere dos pasos explícitos.
Alto riesgo de fugas de memoria o corrupción si se interrumpe.

AUTOMÁTICO



Usa context managers.
Seguro, limpio y cierra automáticamente incluso ante errores.

**Un archivo abierto bloquea recursos en la RAM.
Siempre debe cerrarse.**

EL MÉTODO MANUAL

```
f = open("contactos.txt", "r")
datos = f.read()

# Operaciones...

f.close() ← 
```

Requiere el uso explícito de `open()` y `close()`. Si el programa falla antes del `close()`, el archivo queda corrupto o bloqueado.

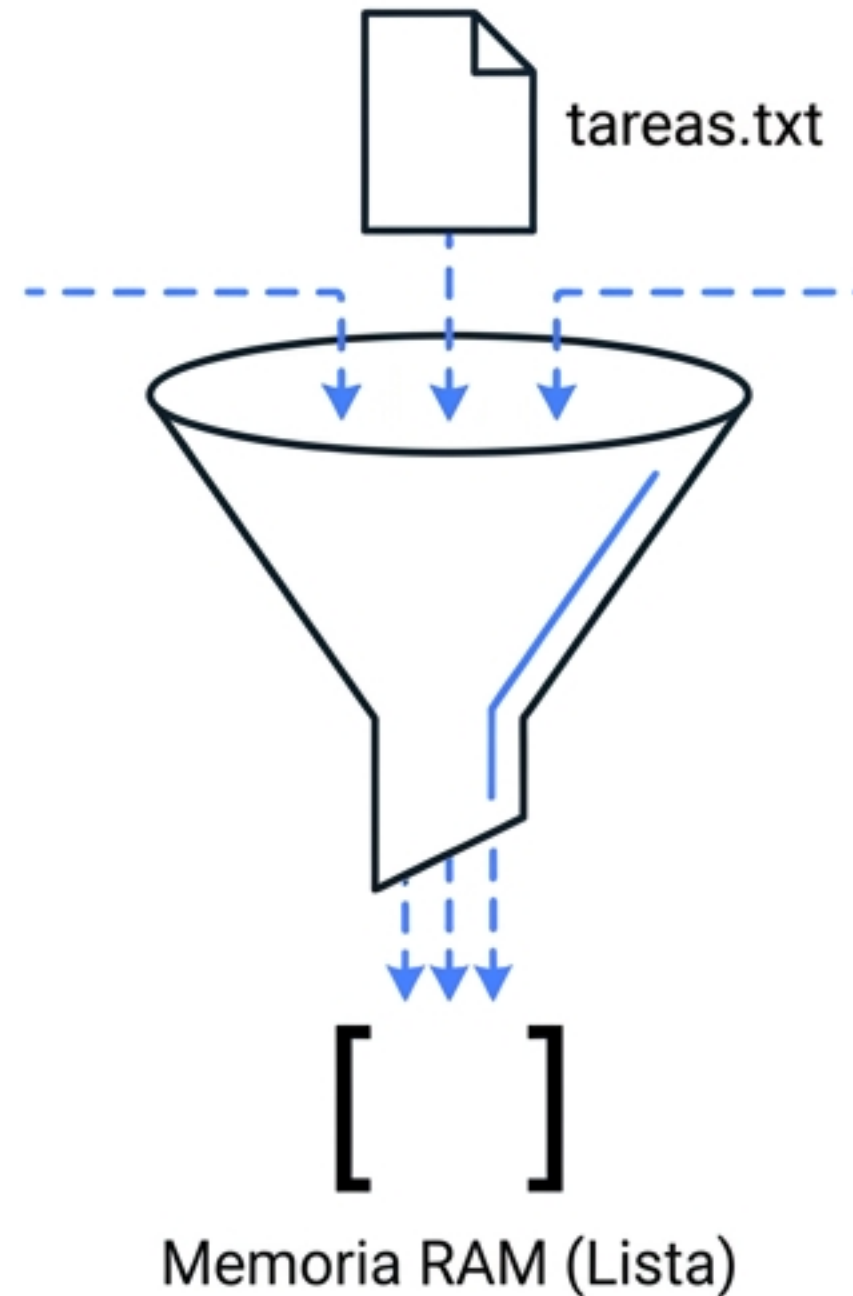
EL MÉTODO AUTOMÁTICO (RECOMENDADO)

```
with open("contactos.txt", "r") as f:  
    datos = f.read()  
    # El archivo se cierra automáticamente al salir de la indentación
```



Utiliza el bloque `with`. El archivo se cierra solo, garantizando la seguridad de los datos. Cero fugas de memoria.

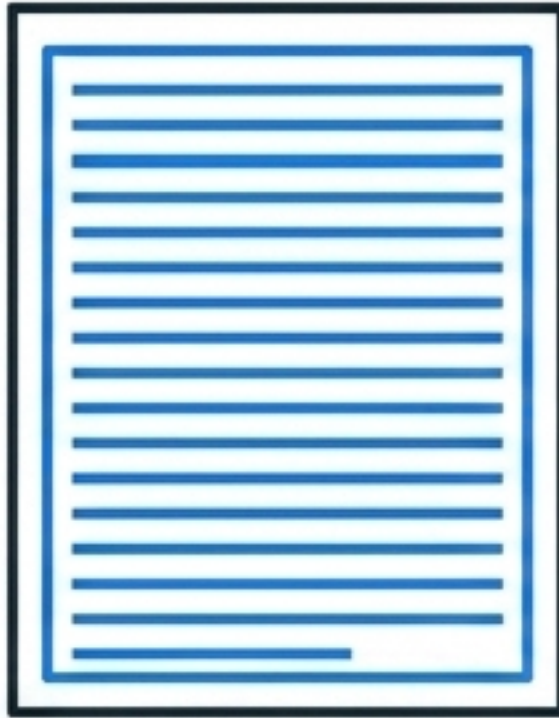
CÓMO ACCEDER A UN FICHERO



Acceder implica leer. Transferimos los datos estáticos del disco a una **estructura dinámica** (como una Lista o Diccionario) en la **memoria RAM** para poder programar con ellos.

MÉTODOS DE LECTURA

read()



Retorna un único
String gigante.

readline()



Retorna un String a la
vez (línea por línea).

readlines()



Retorna una Lista
de Strings.

Elige el método según cómo necesites manipular los datos en RAM.
readlines() es el más versátil para bases de datos de texto.

EJEMPLO: LECTURA SENCILLA

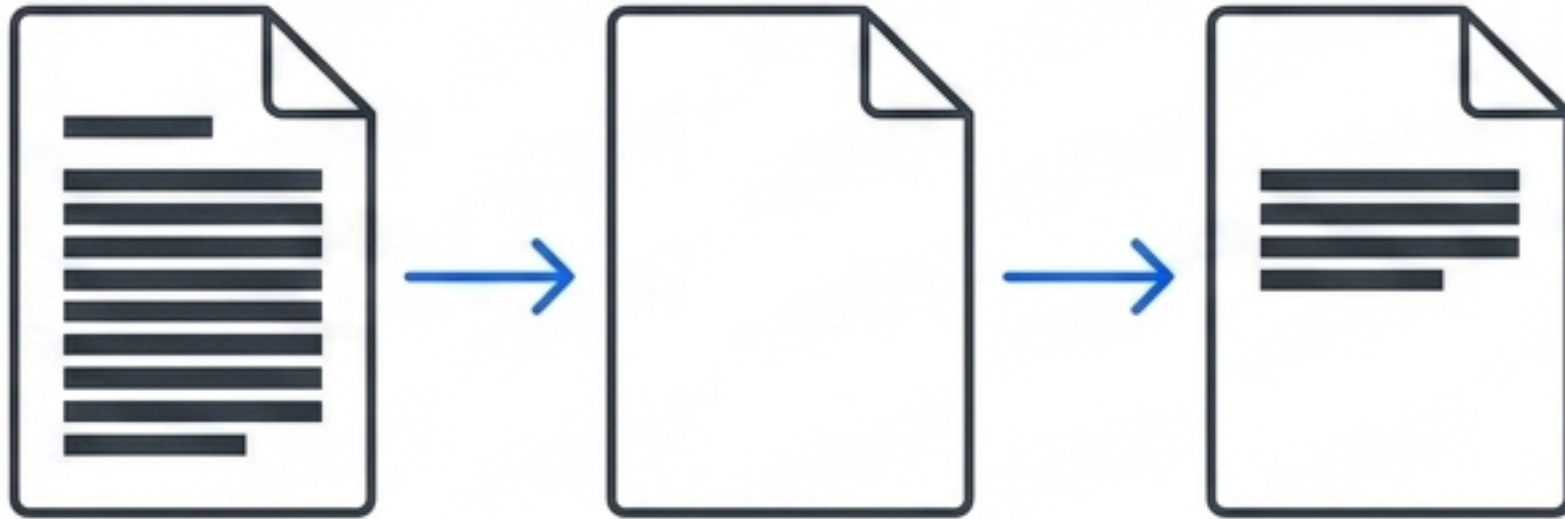
```
with open("tareas.txt", "r") as f:  
    lineas = f.readlines()  
    print(lineas[0])
```

```
> _  
  
> '1; 10-05-2024, 10:30; Enviar  
instrucciones; Registrada\n'
```

Usamos el modo 'r' y **with**. La lista resultante nos permite acceder a cualquier línea mediante su índice.

CÓMO MODIFICAR UN FICHERO

Sobreescritura Destructiva (Modo 'w')

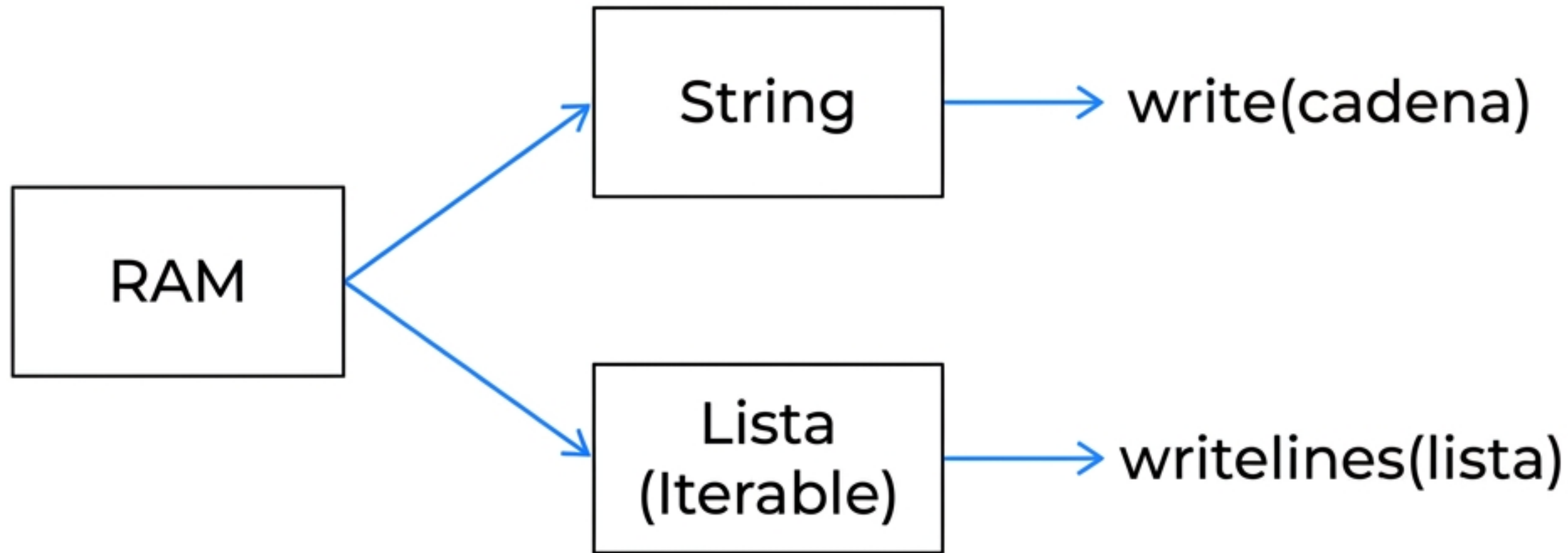


Añadir al Final (Append) (Modo 'a')



Modificar un archivo entero requiere escribir en él.
Ten cuidado: el modo 'w' destruye todo el contenido previo al abrir.

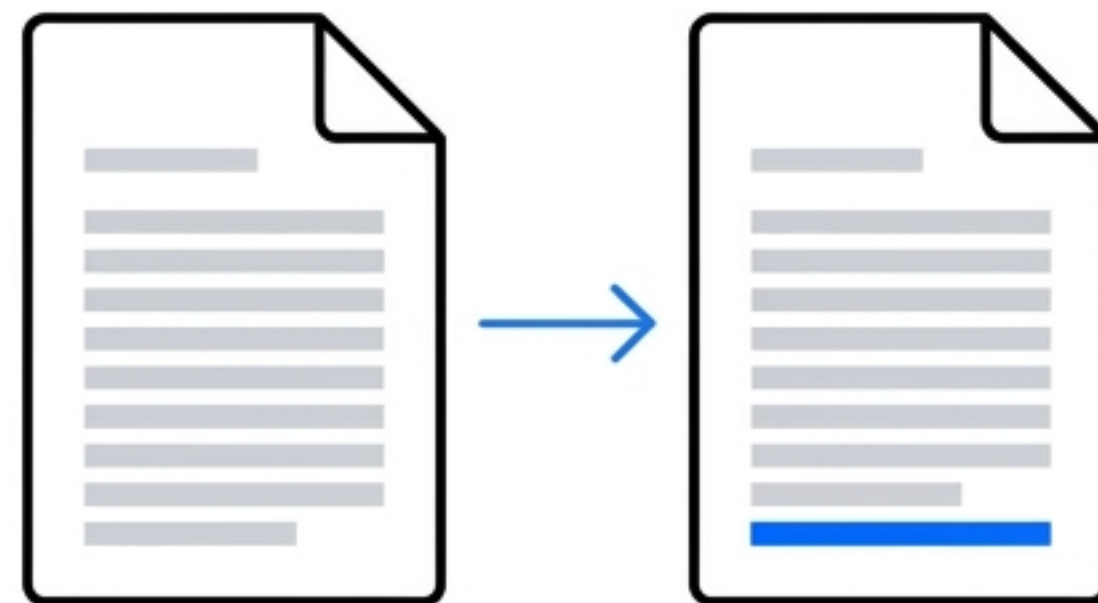
MÉTODOS DE ESCRITURA



write() requiere texto plano. **writelines()** requiere un iterable (como una lista de strings). Ninguno agrega saltos de línea (**\n**) automáticamente; debes incluirlos.

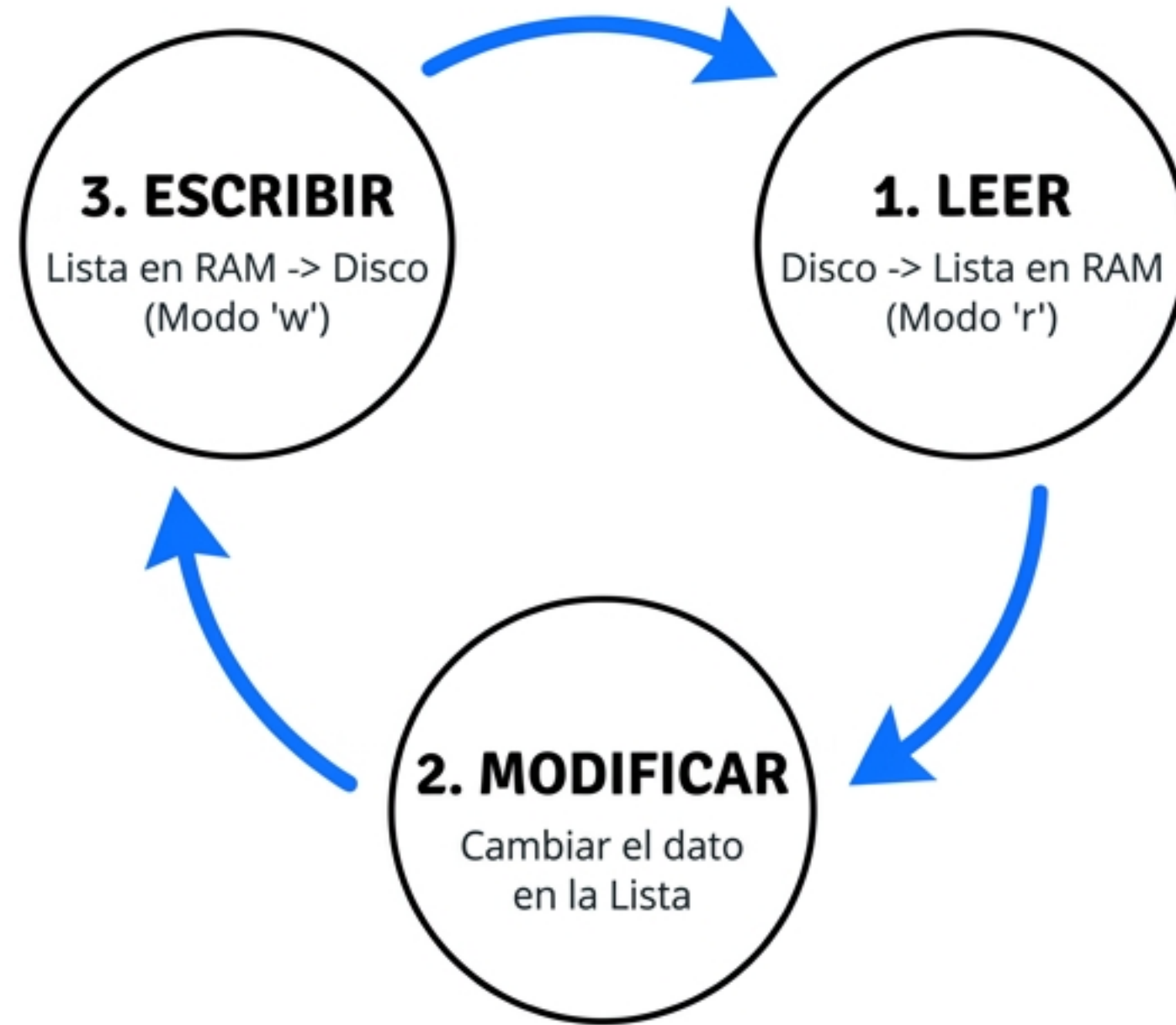
EJEMPLO: ESCRITURA SENCILLA

```
nuevo = "Carlos, 3889999999, carlos@gmail.com\n"  
with open("contactos.txt", "a") as f:  
    f.write(nuevo)
```



El modo **'a'** es perfecto para registrar nuevos eventos, tareas o contactos sin afectar la información existente.

CÓMO MODIFICAR PARTES DE UN FICHERO



Los archivos de texto son secuenciales. No puedes insertar en el medio. Debes cargar todo a la memoria, modificar la memoria y sobrescribir el disco por completo.

EL FLUJO DE ACTUALIZACIÓN



**La RAM es tu mesa de trabajo rápida y volátil.
El Disco es tu archivo permanente.**

EJEMPLO: MODIFICAR UN REGISTRO

```
# 1. Leer todo a una lista
with open("tareas.txt", "r") as f:
    lineas = f.readlines()
```



```
# 2. Modificar en RAM (ej: línea 0)
lineas[0] = "1; 10-05-2024; Revisar correos; Completada\n"
```



```
# 3. Sobrecribir todo
with open("tareas.txt", "w") as f:
    f.writelines(lineas)
```

Al usar 'w' en el paso 3, reemplazamos el archivo viejo con la lista actualizada que tenemos en RAM.

CÓMO ELIMINAR PARTES DE UN FICHERO



Eliminar una línea físicamente requiere el mismo proceso:
extraes todo, filtras lo que no quieres y guardas el resto.

EJEMPLO: ELIMINAR UN REGISTRO

```
with open("contactos.txt", "r") as f:
    lineas = f.readlines()

with open("contactos.txt", "w") as f:
    for linea in lineas:
        if "María" not in linea: # Filtro
            f.write(linea)
```

Al iterar sobre la lista en RAM, escribimos de vuelta al disco todo excepto el registro que deseamos eliminar.

CÓMO ELIMINAR UN FICHERO COMPLETO

```
import os

if os.path.exists("tareas_viejas.txt"):
    os.remove("tareas_viejas.txt")
```



Para **borrar** el archivo físico del disco duro, Python delega la acción al Sistema **Operativo** mediante la librería **os**. La acción es irreversible.

RESUMEN: MÉTODOS NECESARIOS

Apertura	<code>open(), with open() as</code>
Lectura	<code>read()[str], readline()[str], readlines()[list]</code>
Escritura	<code>write(str), writelines(list)</code>
Sistema	<code>import os, os.remove()</code>

REGLAS DE ORO



Usa with open.

(Nunca olvidarás cerrar).



Define tu modo.

(r para leer, w para borrar/escribir, a para agregar).



RAM es la mesa, Disco es el archivo.

(Patrón Leer-Modificar-Escribir para cambios parciales).

Dominar estas tres reglas garantiza una gestión de ficheros segura, eficiente y libre de corrupción de datos.